

Comprehensive Implementation Framework for the AI-Enhanced SOAR Pipeline

1. Introduction and Architectural Paradigm

The contemporary cybersecurity landscape is characterized by a rapidly expanding attack surface and an unprecedented proliferation of sophisticated threat vectors. Security Operations Centers (SOCs) are tasked with monitoring enterprise IT infrastructures, analyzing network traffic, and responding to anomalies detected by Security Information and Event Management (SIEM) systems and Endpoint Detection and Response (EDR) platforms.¹ However, the sheer volume of telemetry generated by these security controls frequently exceeds human cognitive capacity, resulting in a phenomenon known as alert fatigue.¹ Analysts are overwhelmed by thousands of daily alerts, a significant portion of which are benign anomalies or false positives. This cognitive overload leads to increased Mean Time to Detect (MTTD) and Mean Time to Respond (MTTR), thereby increasing the risk that critical intrusions go unnoticed.²

To systematically dismantle this operational bottleneck, the AI-Enhanced Security Orchestration, Automation, and Response (SOAR) platform—codenamed ARIES—integrates a sophisticated artificial intelligence pipeline designed to automate alert triage, orchestrate adaptive response workflows, and accelerate incident investigation through advanced Natural Language Processing (NLP).² The integration of artificial intelligence into the SOAR architecture fundamentally shifts the operational paradigm from a reactive, manual triage process to a predictive, automated defense mechanism.⁴

The deployment of this AI pipeline demands a highly robust, scalable, and decoupled microservices architecture. The system must process an ingestion rate of up to 500 alerts per second while maintaining a median end-to-end processing latency of under two seconds.² Achieving these stringent Service Level Agreements (SLAs) necessitates the integration of distributed message brokers, high-performance computing (HPC) for model training, containerized MLOps tracking environments, and highly optimized inference engines.²

The architectural blueprint of the AI pipeline is structurally partitioned into four interdependent domains. First, the ML Triage Engine functions as a supervised classification system that evaluates normalized alert payloads to predict the True-Positive Likelihood of a security event.² Second, the NLP Intelligence Services deploy transformer-based architectures to parse unstructured textual data, executing Named Entity Recognition (NER) to extract Indicators of Compromise (IOCs) and generating concise executive summaries.² Third, the Continuous Retraining Loop establishes a feedback-driven MLOps infrastructure that captures

human-in-the-loop (HITL) decisions to continuously fine-tune the models against concept drift.¹ Finally, the Low-Latency Inference API serves the optimized models to the broader SOAR ecosystem utilizing asynchronous, event-driven networking protocols.²

This comprehensive report details the end-to-end technical implementation of this AI pipeline. It exhaustively delineates the strategic acquisition of open-source cybersecurity datasets, the execution of distributed deep learning training on the VUB Hydra HPC cluster, the instantiation of a localized, zero-cost MLOps infrastructure using Docker Compose, the mathematical optimization of models via the Open Neural Network Exchange (ONNX) format, and the asynchronous integration of the inference engine into the platform's Apache Kafka event bus.

2. Strategic Dataset Acquisition and Feature Engineering

The predictive accuracy, generalization capability, and operational reliability of the AI pipeline are inextricably linked to the quality, volume, and relevance of its foundational training data. Because the ARIES platform must perform both tabular data classification for initial alert triage and complex unstructured textual analysis for incident summarization, two distinct categories of datasets must be sourced, engineered, and integrated.² The development strategy strictly relies on authoritative, open-source datasets that provide heavily annotated, real-world security telemetry, thereby avoiding the fragility and bias inherent in purely synthetic data generation.⁶

2.1 Tabular Data for the ML Triage Engine: The Microsoft GUIDE Dataset

To train machine learning models capable of accurately assigning a True-Positive Likelihood score to incoming SIEM and EDR alerts, the system requires a massive corpus of historical SOC data encompassing diverse attack vectors and benign anomalies. The Microsoft GUIDE (Guided Response) Dataset serves as the optimal foundational resource for this classification task.⁶

Released by Microsoft Security Research under the permissive CDLA-2.0 license, the GUIDE dataset is currently the largest publicly available collection of real-world cybersecurity incidents.⁶ Hosted on platforms such as Kaggle, it provides a highly stratified, hierarchical view of enterprise security operations across a vast array of organizational environments.⁶ The dataset is explicitly designed to support the development of AI-driven guided response systems that facilitate informed decision-making by SOC analysts.⁶

Dataset Characteristic	Specification Details
------------------------	-----------------------

Total Volume and Scope	Comprises over 13 million individual pieces of evidence, aggregated into 1.6 million alerts, and further synthesized into 1 million distinct security incidents spanning a two-week observation period. ⁶
Telemetry Breadth	Incorporates security telemetry from over 6,100 distinct organizations, utilizing 9,100 unique custom and built-in detectors (DetectorIds) across numerous security products. ⁶
Framework Alignment	The recorded incidents encompass 441 distinct techniques documented within the MITRE ATT&CK framework, ensuring comprehensive coverage of modern adversary behaviors. ⁶
Annotation Methodology	Features ground-truth triage labels generated by professional customer security analysts. The labels are categorized into three primary classes: True Positive (TP), False Positive (FP), and Benign Positive (BP). ⁹
Architectural Structure	Exhibits a three-level hierarchical structure: Evidence (atomic data points like IP addresses), Alerts (aggregated evidence signaling a potential threat), and Incidents (collections of alerts representing a complete scenario). ⁹
Data Separation	Pre-configured with a 70/30 train-test split, maintaining strict stratification by triage grade, organization ID, and detector ID to prevent data leakage during model evaluation. ¹¹

2.1.1 Feature Engineering and Class Imbalance Mitigation

The raw telemetry provided by the Microsoft GUIDE dataset presents specific machine learning challenges that must be addressed through rigorous feature engineering before model ingestion. The most prominent challenge is severe class imbalance. In real-world enterprise

environments, a massive proportion of generated alerts are benign or false positives, resulting from misconfigured systems or authorized administrative actions.¹ Training a classification algorithm directly on this skewed distribution will invariably yield a biased model that heavily favors the majority class, consequently failing to identify rare but catastrophic True Positives.

To counteract this statistical bias, the data preprocessing pipeline must implement resampling techniques. While simple random oversampling can lead to overfitting, the Synthetic Minority Over-sampling Technique (SMOTE) generates synthetic examples of the minority classes (TPs) by interpolating between existing instances in the feature space, thereby enriching the decision boundary.⁹ Alternatively, cost-sensitive learning can be implemented by adjusting the class weights dynamically during the computation of the loss function, heavily penalizing the model for misclassifying actual threats.⁹

Furthermore, the dataset contains high-cardinality categorical features, such as DetectorId and OrgId.¹¹ Traditional one-hot encoding of these features would result in an explosion of the feature space dimensionality, leading to the curse of dimensionality and severely degrading model inference latency. Instead, these categorical variables must be transformed using target encoding or learned embedding layers, which map the discrete categories into dense, lower-dimensional continuous vectors that capture the latent semantic relationships between different detection mechanisms.

The ultimate target variable for the ML Triage Engine is the continuous True-Positive Likelihood score required by the SOAR orchestration layer.² By modeling the output as a calibrated probability distribution over the TP, BP, and FP classes using algorithms such as Gradient Boosting (e.g., XGBoost, LightGBM) or deep feed-forward neural networks, the system can extract a precise confidence metric ranging from 0.0 to 1.0, enabling the orchestrator to route the alert automatically or escalate it for human review.²

2.2 Textual Data for NLP Services: The CTISum Dataset

While tabular data drives the triage routing engine, the ARIES platform simultaneously deploys an NLP module tasked with parsing unstructured, verbose threat data to extract critical entities and generate cohesive incident summaries.² Security analysts frequently spend significant portions of their operational time reading through lengthy malware analysis reports, raw log files, and threat intelligence feeds to understand the context of an attack.¹⁴ Automating this comprehension requires training a language model on a highly specialized, domain-specific cybersecurity lexicon.

The CTISum (Cyber Threat Intelligence Summarization) dataset provides the requisite textual data structure.¹⁵ Developed explicitly to benchmark the capabilities of generative language models within the cybersecurity domain, CTISum addresses the historical scarcity of high-quality, annotated cyber threat intelligence reports.¹⁶

Dataset Characteristic	Specification Details
Source Material Acquisition	Comprises 1,345 high-quality documents meticulously parsed from complex PDF structures. Sources include open-source APT (Advanced Persistent Threat) notes, threat encyclopedias, and detailed incident reports from major security vendors such as Symantec and TrendMicro. ¹⁵
Core Task 1: General Summarization	Designed to train abstractive summarization models to condense lengthy, technical threat reports into concise, executive-level overviews, highlighting the most salient facts and analytical insights. ¹⁵
Core Task 2: Attack Process Summarization (APS)	Introduces a novel, fine-grained subtask focusing exclusively on extracting and summarizing the sequential, chronological steps of a cyber attack vector, enabling defenders to rapidly assess risks and identify exploited vulnerabilities. ¹⁶
Repository Accessibility	The dataset, along with associated baseline models and evaluation scripts (utilizing metrics like ROUGE-n and BERTScore), is hosted on GitHub under the repository pengwei-iiie/CTISum. ¹⁶

2.2.1 NLP Training Objectives and Data Structuring

The CTISum dataset facilitates the fine-tuning of pre-trained transformer architectures. The primary training objective for the entity extraction pipeline involves sequence labeling (token classification). The model is trained to process a sequence of text and assign a categorical label to each token, identifying specific Indicators of Compromise (IOCs) such as malicious IP addresses, cryptographic file hashes, specific malware families, and known threat actor aliases.¹⁴ This structured extraction allows the SOAR platform to automatically cross-reference these entities against external threat intelligence feeds via the Integration Connectors.²

The secondary training objective addresses abstractive summarization. Unlike extractive summarization, which merely copies existing sentences from the source text, abstractive

summarization trains sequence-to-sequence (Seq2Seq) models (e.g., BART, T5) to generate entirely new, coherent narratives that encapsulate the core meaning of the original document.¹⁶ This capability drastically reduces the cognitive load on SOC analysts, providing them with immediate, actionable intelligence upon opening an incident case.²

3. Distributed Model Training on the VUB Hydra HPC

The training phase for deep learning models—particularly large parameter transformer architectures for NLP tasks and complex neural networks for tabular classification—is computationally prohibitive on standard commercial hardware. Training these models on millions of data points requires specialized hardware accelerators to perform parallel matrix multiplications. The computational resources provided by the Vrije Universiteit Brussel (VUB) Hydra High-Performance Computing (HPC) cluster fulfill this critical infrastructure requirement.²²

Hydra operates as a Tier-2 heterogeneous batch cluster, maintained under a policy of permanent upgrades.²³ It provides exclusive computational resources distributed via the Slurm Workload Manager, ensuring that heavily parallelized machine learning jobs run in isolated, dedicated environments without resource contention.²²

3.1 HPC Hardware Specifications and Optimal Partition Selection

To maximize training velocity and minimize queue wait times, jobs must be intelligently routed to the appropriate hardware partitions.²⁵ The Hydra cluster comprises several specialized GPU partitions engineered for high-throughput deep learning tasks.

Slurm Partition	Hardware Configuration	Memory Specifications	Network Interconnect
ampere_gpu	2x NVIDIA A100 (Ampere) GPUs per node. Dual 16-core AMD EPYC 7282 processors.	40 GB VRAM per GPU. 256 GB system RAM per node.	100 Gbps EDR InfiniBand for low-latency node communication. ²³
hopper_gpu	2x NVIDIA H200 (Hopper) GPUs per node. Dual 24-core AMD EPYC 9275F processors.	140 GB VRAM per GPU. 754 GB system RAM per node.	400 Gbps NDR InfiniBand. ²³

pascal_gpu	2x NVIDIA Tesla P100 (Pascal) GPUs per node. Dual 12-core Intel Xeon processors.	16 GB VRAM per GPU. 256 GB system RAM per node.	10 Gbps standard ethernet network. ²³
------------	--	---	--

For the fine-tuning of transformer models on the CTISum dataset, memory bandwidth is the primary bottleneck. Large language models require substantial Video RAM (VRAM) to store model weights, gradients, optimizer states, and activations during the forward and backward passes. Consequently, targeting the ampere_gpu partition is the optimal strategic choice. The 40 GB of VRAM provided by the A100 GPUs enables the use of larger batch sizes and sequence lengths, which stabilizes the loss gradients and significantly accelerates convergence rates compared to older hardware generations.²³

3.2 Ephemeral Environment Configuration and Dependency Management

The VUB Hydra HPC strictly enforces a clean execution environment. Upon the initiation of any Slurm job, the system \$PATH is completely reset, and no software modules are loaded into the environment by default.²² This design philosophy guarantees absolute scientific reproducibility, ensuring that a job executed today will perform identically in the future without interference from hidden system dependencies. However, it necessitates explicit initialization of the software stack within every submission script.²²

While Hydra provides a vast repository of pre-compiled software accessible via the hierarchical module load command²⁶, modern deep learning pipelines often require highly specific, tightly coupled library versions. For instance, PyTorch must be exactly matched with specific NVIDIA CUDA runtime versions, and the Hugging Face transformers ecosystem frequently requires bleeding-edge updates that may not yet exist in the cluster's global module tree.²⁸

The most robust approach to managing these dependencies is to bypass the global module system in favor of constructing isolated, user-defined virtual environments using Conda (facilitated by the Mamba module for accelerated dependency resolution).²⁸ Crucially, these environments and the associated datasets must be installed directly onto the cluster's high-speed \$VSC_SCRATCH storage system.³¹ Writing millions of small files (like Python library binaries) to the standard \$HOME directory rapidly exhausts inode quotas and severely degrades I/O performance over the network file system.³¹

To initialize the deep learning environment interactively prior to executing a batch training run, the following procedure is executed:

Bash

```
# Request an interactive CPU session to build the environment without consuming GPU quota  
srun --nodes=1 --ntasks=1 --cpus-per-task=4 --time=01:00:00 --pty bash -l
```

```
# Load the Mamba module from the global system  
module load Mamba
```

```
# Create the isolated environment on the high-speed scratch disk  
conda create --prefix $VSC_SCRATCH/aries_env python=3.11  
source $EBROOTMAMBA/etc/profile.d/conda.sh  
conda activate $VSC_SCRATCH/aries_env
```

```
# Install PyTorch with specific CUDA bindings and Hugging Face libraries  
conda install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia  
pip install transformers datasets evaluate optimum[exporters] aiokafka
```

3.3 Constructing the Slurm Batch Script for Distributed Training

Once the isolated environment is fully configured, the model training process is codified into a declarative Slurm batch script. This script acts as a manifest, requesting the exact physical resources required from the HPC scheduler and defining the execution steps.²²

For the NLP summarization task using the CTISum dataset, a single GPU is often insufficient to process the data within a reasonable timeframe. The architecture must scale horizontally. Utilizing Hugging Face's Accelerate library, the training loop can be seamlessly distributed across multiple GPUs within a single node without requiring the manual, complex initialization of PyTorch's Distributed Data Parallel (DDP) backend.³²

The following Slurm batch script illustrates the configuration for a multi-GPU training job on the `ampere_gpu` partition:

Bash

```
#!/bin/bash  
#SBATCH --job-name=aries_nlp_finetune  
#SBATCH --time=24:00:00  
#SBATCH --partition=ampere_gpu  
#SBATCH --nodes=1  
#SBATCH --gpus-per-node=2
```

```

#SBATCH --ntasks-per-gpu=1
#SBATCH --cpus-per-task=16
#SBATCH --mem=128G
#SBATCH --output=%x-%j.out
#SBATCH --error=%x-%j.err

# Ensure clean environment initialization
module purge
module load Mamba
source $EBROOTMAMBA/etc/profile.d/conda.sh
conda activate $VSC_SCRATCH/aries_env

# Navigate to the project directory on the scratch disk
cd $VSC_SCRATCH/aries_project/

# Launch the distributed training script using Accelerate
accelerate launch \
  --multi_gpu \
  --num_processes=2 \
  --mixed_precision=fp16 \
  train_nlp_summarization.py \
  --dataset_path./data/CTISum \
  --model_name_or_path facebook/bart-large \
  --output_dir./models/bart_finetuned

```

In this configuration, `--gpus-per-node=2` allocates both A100 GPUs on the physical server, while `--cpus-per-task=16` ensures that the data loaders have sufficient CPU bandwidth to preprocess the text batches without starving the GPUs of data.²³ The `--mixed_precision=fp16` flag within the `accelerate launch` command instructs the GPUs to perform calculations using 16-bit floating-point numbers where mathematically safe, doubling the throughput and halving the memory footprint compared to standard 32-bit precision.³²

4. Containerized MLOps Infrastructure

Training machine learning models on a high-performance cluster represents only the initial experimental phase of the AI lifecycle. Transitioning these models from research artifacts into reliable, production-grade microservices mandates the implementation of a rigorous Machine Learning Operations (MLOps) framework.² The ARIES platform requires a systematic infrastructure to meticulously track experimental runs, log hyperparameter configurations, store versioned model weight artifacts, and facilitate seamless model promotion and rollback procedures.²

MLflow serves as the definitive, industry-standard tool for fulfilling these requirements.³⁵ While

cloud-managed instances of MLflow exist (e.g., via Databricks or AWS SageMaker), the architectural directives of the ARIES project stipulate the utilization of free or low-cost tools.³⁶ Furthermore, deploying the MLOps stack locally using containerization ensures that highly sensitive cybersecurity telemetry and proprietary model weights do not inadvertently traverse the public internet, thereby maintaining strict data sovereignty and organizational perimeter security.

4.1 MLOps Architecture Components

A production-ready, self-hosted MLflow deployment eschews the simplistic, default file-based storage in favor of a robust, three-tiered service architecture³⁷:

1. **MLflow Tracking Server:** A lightweight, FastAPI-based Python web application that serves as the central API gateway and interactive user interface. It receives and processes REST API logging requests generated by the training scripts.³⁷
2. **Backend Store (PostgreSQL):** A highly structured relational database responsible for persistently storing all experiment metadata. This includes quantitative metrics (e.g., F1-scores, training loss curves), qualitative parameters, tags, and historical run data.³⁷ The default SQLite backend is wholly inadequate for this role, as it suffers from severe database locking issues in concurrent, multi-run environments and lacks the scalability required for a multi-analyst platform.³⁷ PostgreSQL ensures ACID compliance and data integrity across thousands of experimental iterations.³⁹
3. **Artifact Store (MinIO):** A high-performance, S3-compatible object storage server. While relational databases excel at storing metadata, they are highly inefficient at storing massive binary blobs. MinIO assumes responsibility for storing large artifacts, such as the serialized PyTorch .pt model weights, exported ONNX computational graphs, and generated evaluation plots.³⁷ Because MinIO replicates the exact API of Amazon S3, the underlying code remains entirely cloud-agnostic, allowing for a frictionless transition to managed cloud storage if the platform scales beyond local capacity.³⁷

4.2 Docker Compose Orchestration and Networking

The seamless integration and orchestration of these three distinct services are managed via a single docker-compose.yml configuration file.³⁶ This approach encapsulates the entire MLOps environment, ensuring perfect parity between development, testing, and production deployments.²

The Docker Compose configuration establishes an isolated, internal bridge network (ml_network). This prevents the backend database and object store from exposing their ports to the host machine's public interfaces, routing all external traffic exclusively through the MLflow Tracking Server API.³⁷

YAML

version: '3.8'

services:

postgres_db:

image: postgres:14

container_name: aries_mlflow_db

restart: always

environment:

POSTGRES_USER: \${POSTGRES_USER}

POSTGRES_PASSWORD: \${POSTGRES_PASSWORD}

POSTGRES_DB: \${POSTGRES_DB}

volumes:

- postgres_data:/var/lib/postgresql/data

networks:

- ml_network

minio_store:

image: minio/minio

container_name: aries_mlflow_artifacts

restart: always

ports:

- "9000:9000"

- "9001:9001"

environment:

MINIO_ROOT_USER: \${MINIO_ACCESS_KEY}

MINIO_ROOT_PASSWORD: \${MINIO_SECRET_KEY}

command: server /data --console-address ":9001"

volumes:

- minio_data:/data

networks:

- ml_network

mlflow_server:

image: ghcr.io/mlflow/mlflow:latest

container_name: aries_mlflow_server

restart: always

ports:

- "5000:5000"

environment:

```

    AWS_ACCESS_KEY_ID: ${MINIO_ACCESS_KEY}
    AWS_SECRET_ACCESS_KEY: ${MINIO_SECRET_KEY}
    MLFLOW_S3_ENDPOINT_URL: http://minio_store:9000
command: >
    mlflow server
    --backend-store-uri
postgresql://${POSTGRES_USER}:${POSTGRES_PASSWORD}@postgres_db:5432/${POSTGRES_DB}
    --default-artifact-root s3://aries-models/
    --host 0.0.0.0
    --port 5000
depends_on:
    - postgres_db
    - minio_store
networks:
    - ml_network

volumes:
    postgres_data:
    minio_data:

networks:
    ml_network:
        driver: bridge

```

To maintain security hygiene, all sensitive credentials (database passwords, S3 access keys) are injected dynamically at runtime via an untracked .env file, ensuring that cryptographic secrets are never hardcoded into the version control repository.³⁹

4.3 Resolving the HPC-to-Local Synchronization Challenge

A critical integration challenge materializes when training models on the isolated VUB Hydra HPC cluster while maintaining the MLflow tracking server on a local, localized Docker host. HPC compute nodes are situated behind strict firewalls and typically restrict outbound internet access to non-standard ports (such as port 5000 used by MLflow).⁴² Consequently, the PyTorch training scripts executing on the HPC nodes cannot establish a direct HTTP connection to send real-time telemetry back to the local tracking server.

To circumvent this absolute networking constraint, the pipeline employs an offline synchronization strategy. Within the Python training script executing on Hydra, the MLflow tracking URI is not directed to a remote IP address; instead, it is explicitly configured to target a local directory residing on the high-speed scratch disk⁴²:

Python

```
import mlflow
# Configure MLflow to write tracking data directly to the local HPC filesystem
mlflow.set_tracking_uri("file:///scratch/vsc10000/aries_project/mlruns")
```

Throughout the training epoch, MLflow writes all metrics, parameters, and serialized model artifacts to this localized mlruns directory. Upon the successful completion of the Slurm job, this directory contains a complete, immutable record of the experiment. The directory is subsequently transferred back to the local machine via secure copy protocols (scp or rsync). Once residing on the local filesystem, the data can be systematically ingested into the PostgreSQL and MinIO Docker stack, ensuring that the central tracking server retains an exhaustive history of all HPC experiments without violating the cluster's stringent security perimeters.⁴²

5. Model Optimization and Quantization for Low-Latency Inference

The overarching mandate of the ARIES SOAR platform is operational velocity. The system is engineered to absorb massive bursts of telemetry, requiring the capacity to process an ingestion rate of at least 500 alerts per second while strictly maintaining an end-to-end processing latency of under two seconds per alert.² Deploying raw, unoptimized PyTorch models directly into the production inference pipeline fundamentally threatens these Service Level Agreements (SLAs).

While PyTorch is exceptional for dynamic computational graph construction and rapid prototyping during the research and training phases, it carries significant overhead during inference. The Python Global Interpreter Lock (GIL) inherently limits true multi-threaded concurrency, forcing requests to process sequentially rather than in parallel.⁴³ Furthermore, native PyTorch execution paths are heavily bloated with autograd tracking mechanisms and are not universally optimized for varying hardware targets.⁴³

The strategic solution to this bottleneck is the compilation and exportation of the trained models into the Open Neural Network Exchange (ONNX) format, subsequently utilizing the ONNX Runtime (ORT) as the foundational engine for the serving layer.²

5.1 The Architecture and Advantages of ONNX Runtime

ONNX provides a standardized, interoperable intermediate representation (IR) of the model's underlying computational graph.⁴⁵ By exporting the PyTorch model to ONNX, the inference engine is entirely decoupled from the deep learning framework originally used to train it. The

ONNX Runtime engine is implemented in highly optimized C++ and completely bypasses the restrictive Python GIL, enabling genuine multi-threaded, concurrent inference across available CPU cores.⁴³

Furthermore, ORT introduces the concept of Execution Providers. This allows the exact same ONNX model file to seamlessly leverage vastly different hardware accelerators—ranging from standard Intel CPUs to NVIDIA CUDA cores or dedicated TensorRT engines—without requiring any alterations to the deployment application code.⁴⁶

5.2 Exporting Models via Hugging Face Optimum CLI

Hugging Face provides the optimum library, an extension designed specifically to bridge the complex architectures of transformer models with the optimization capabilities of ONNX.⁴⁷ The conversion process leverages the optimum-cli tool, which automates the highly complex task of tracing the model's forward pass and constructing the static ONNX graph.

Following the successful training of the ML Triage model on the Microsoft GUIDE dataset, the raw PyTorch artifact is exported directly to an optimized ONNX graph using the following terminal command⁴⁷:

Bash

```
optimum-cli export onnx \  
  --model./trained_triage_model \  
  --task text-classification \  
  --optimize O2 \  
  --device cpu \  
  ./onnx_triage_model/
```

The --optimize O2 parameter is of paramount architectural importance. It instructs the ONNX graph optimizer to perform complex mathematical fusions that are specifically tailored for transformer topologies.⁴⁹ In a standard PyTorch execution, operations such as Layer Normalization, Multi-Head Attention, and GELU activations are executed sequentially, requiring the GPU or CPU to read and write intermediate tensors to memory repeatedly. The O2 optimization fuses these consecutive mathematical operations into single, highly efficient computational kernels.⁴⁹ This drastically reduces memory bandwidth bottlenecks, which are typically the primary limiting factor in transformer inference speed.⁴⁹

For deployments targeting specialized GPU-accelerated inference nodes, the O4 optimization level can be utilized. This level applies mixed-precision (FP16) quantization, converting the

32-bit floating-point weights into 16-bit representations.⁴⁹ This effectively halves the memory footprint of the model and doubles the execution throughput, with a mathematically negligible impact on the model's predictive accuracy.⁴⁹

6. High-Performance Inference Microservice Development

With the highly optimized ONNX computational graphs generated, the subsequent architectural phase is the construction of the specific microservice responsible for serving predictions to the broader SOAR ecosystem. FastAPI is selected as the definitive framework for this critical ingestion layer.²

FastAPI is a modern, exceptionally high-performance web framework for building APIs with Python. It natively implements the Asynchronous Server Gateway Interface (ASGI) standard via the Uvicorn server, supporting fully asynchronous programming (async/await).⁵ This architectural trait is vital for handling thousands of concurrent network connections without blocking the application thread.⁵ When coupled with Pydantic for rigorous data schema validation, FastAPI ensures that malformed or malicious alert payloads cannot crash the inference engine.⁵⁰

6.1 Architectural Design of the FastAPI Service

The inference service is designed as a containerized, stateless application. A fundamental design pattern to minimize latency involves loading the massive ONNX model graphs into memory exactly once during the application's initialization phase.⁴⁴ If the model were loaded from disk on every incoming request, the massive I/O overhead would render the sub-second latency SLA impossible to achieve.⁴⁴

Python

```
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
import onnxruntime as ort
from transformers import AutoTokenizer
import numpy as np
```

```
app = FastAPI(title="ARIES ML Triage Inference Service")
```

```
# Global variables to hold the instantiated model session and tokenizer
```

```

session = None
tokenizer = None

@app.on_event("startup")
async def load_model_assets():
    global session, tokenizer
    # Load the optimized ONNX graph into memory using the CPU Execution Provider
    session = ort.InferenceSession("onnx_triage_model/model.onnx",
    providers=["CPUExecutionProvider"])
    # Load the corresponding Hugging Face tokenizer mapping
    tokenizer = AutoTokenizer.from_pretrained("onnx_triage_model/")

class AlertPayload(BaseModel):
    alert_id: str
    text_data: str
    # Additional fields omitted for brevity

@app.post("/predict")
async def predict_triage(payload: AlertPayload):
    try:
        # Preprocess the textual data into tokenized numerical arrays
        inputs = tokenizer(payload.text_data, return_tensors="np", truncation=True, padding=True,
        max_length=512)

        # Execute the ONNX inference graph directly
        outputs = session.run(None, {
            "input_ids": inputs["input_ids"],
            "attention_mask": inputs["attention_mask"]
        })

        # Post-process raw logits into bounded probabilities via softmax
        probabilities = calculate_softmax(outputs)

        # Extract the specific probability of the True Positive class
        return {
            "alert_id": payload.alert_id,
            "true_positive_likelihood": float(probabilities)
        }
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Inference computation failed: {str(e)}")

```

6.2 Preprocessing and Memory Management

The FastAPI microservice functions as an orchestrator for the entire data transformation pipeline. Security telemetry arrives as raw, nested JSON payloads.² The API must parse this payload, extract the relevant textual features (as dictated by the schema upon which the Microsoft GUIDE models were trained), and pass them to the NLP tokenizer.⁵

Because the ONNX Runtime engine expects mathematical inputs strictly as highly optimized NumPy arrays (mapping to native C-types), passing the `return_tensors="np"` flag is a strict requirement when invoking the Hugging Face tokenizer.⁴⁴ Following the execution of the ONNX graph, the model returns raw, unbounded logits. These must be mathematically processed through a softmax activation function to output the final `true_positive_likelihood` score, normalizing the output to a range between 0.0 and 1.0. This calculated floating-point metric represents the crucial algorithmic decision node that dictates whether the subsequent Orchestration Engine will automatically close the alert as noise or trigger a complex, multi-system containment playbook.¹

7. Event-Driven Messaging Pipeline Integration

The final structural mandate of the AI pipeline involves the integration of the FastAPI inference service into the broader ARIES platform architecture. A traditional, synchronous REST architecture—where the Ingestion Layer transmits a direct HTTP POST request to the Inference Service and actively waits for a response—is fundamentally fragile at an enterprise scale. If an external security event triggers an ingestion spike of 10,000 simultaneous alerts, a synchronous API will rapidly exhaust its worker pool, resulting in dropped network requests, cascading timeouts, and catastrophic data loss.⁸

To achieve the absolute resilience and throughput required by a modern SOAR platform, the architecture explicitly dictates an asynchronous, event-driven integration pattern, utilizing Apache Kafka as the central, high-throughput message broker.²

7.1 The Asynchronous Messaging Topology

Kafka provides a highly scalable, fault-tolerant, append-only distributed log. The ARIES architecture is fully decoupled into independent producers and consumers that communicate exclusively via specific, structured Kafka topics⁸:

1. **Ingestion & Normalization:** The Logstash ingestion layer parses heterogeneous incoming telemetry from disparate EDR/SIEM tools, standardizes the schema, and acts as a producer, continuously publishing the raw, normalized alerts to the `alerts.raw` Kafka topic.²
2. **Inference Consumption:** The FastAPI microservice acts as an active consumer. It continuously polls the `alerts.raw` topic, pulling messages at its maximum computational capacity. The Kafka broker intrinsically handles the backpressure; if the ingestion rate temporarily exceeds the inference rate (e.g., during a massive network scan), the excess alerts safely queue on the Kafka disk without overwhelming or crashing the FastAPI

application.⁸

3. **Enrichment Production:** Following the successful execution of the ONNX inference graph, the FastAPI service transitions into a producer, publishing the newly scored and enriched alert payload to the alerts.enriched topic.⁸
4. **Orchestration Trigger:** The Go-based Orchestration Engine acts as the final consumer, subscribing to the alerts.enriched topic. Based on the appended risk score, it executes the appropriate adaptive playbook logic.²

7.2 Implementing aiokafka within the FastAPI Event Loop

To integrate Kafka seamlessly within the asynchronous FastAPI event loop without introducing blocking I/O operations, the aiokafka Python library is utilized.⁵⁰ Because aiokafka is built natively on Python's asyncio framework, the continuous polling mechanism of the Kafka consumer does not block the API's ability to serve administrative endpoints, health checks, or manual REST invocations.⁵⁰

The optimal architectural pattern utilizes FastAPI's startup and shutdown event hooks to manage the lifecycle and connection state of the Kafka producers and consumers, ensuring clean connection establishment and teardown.⁵³

Python

```
import asyncio
import json
from fastapi import FastAPI
from aiokafka import AIOKafkaConsumer, AIOKafkaProducer

app = FastAPI(title="ARIES Event-Driven Inference")

KAFKA_BROKER = "kafka:9092"
RAW_TOPIC = "alerts.raw"
ENRICHED_TOPIC = "alerts.enriched"
CONSUMER_GROUP = "ml_triage_group"

# Global state to maintain the persistent producer connection
producer = None

async def consume_and_predict():
    # Initialize consumer with manual offset commit configuration
    consumer = AIOKafkaConsumer(
```

```

    RAW_TOPIC,
    bootstrap_servers=KAFKA_BROKER,
    group_id=CONSUMER_GROUP,
    auto_offset_reset="earliest",
    enable_auto_commit=False
)
await consumer.start()
try:
    async for msg in consumer:
        alert_data = json.loads(msg.value.decode('utf-8'))

        # Execute the ONNX inference logic
        prediction_score = run_inference(alert_data)
        alert_data["triage_score"] = prediction_score

        # Produce the enriched alert back to the downstream topic
        await producer.send_and_wait(
            ENRICHED_TOPIC,
            json.dumps(alert_data).encode('utf-8')
        )

        # Manually commit the offset ONLY after successful production
        await consumer.commit()
finally:
    await consumer.stop()

@app.on_event("startup")
async def startup_event():
    global producer
    # Initialize and start the Kafka Producer
    producer = AIOKafkaProducer(bootstrap_servers=KAFKA_BROKER)
    await producer.start()

    # Launch the Kafka Consumer as an independent, non-blocking background asyncio task
    asyncio.create_task(consume_and_predict())

@app.on_event("shutdown")
async def shutdown_event():
    if producer:
        await producer.stop()

```

7.2.1 Consumer Group Dynamics and Fault Tolerance Semantics

The explicit assignment of a `group_id` (e.g., `ml_triage_group`) within the `AIOKafkaConsumer` configuration is a paramount architectural requirement.⁵¹ This parameter enables true horizontal scalability. If the volume of incoming security alerts necessitates higher inference throughput, systems administrators can deploy additional replica instances of the FastAPI Docker container. Kafka's internal consumer group protocol will automatically detect the new instances and rebalance the partitions of the `alerts.raw` topic across all active containers. This allows parallel inference processing without duplicating work or encountering race conditions.⁵⁵

Furthermore, for strict fault tolerance and data integrity, the `enable_auto_commit` feature must be explicitly disabled.⁵⁵ Instead, the application manually commits the Kafka offset only *after* the enriched alert has been successfully published to the downstream `alerts.enriched` topic.⁵⁵ This configuration guarantees "at-least-once" processing semantics. If the FastAPI container experiences a catastrophic failure or out-of-memory crash mid-inference, the alert offset will not be acknowledged. Upon container restart, the consumer will automatically retrieve and re-process the uncommitted alert, entirely eliminating the risk of silently dropping critical security events during a system fault.⁵⁵

8. Closed-Loop Retraining and Human-in-the-Loop Governance

The integration of the AI pipeline extends significantly beyond initial alert triage and entity extraction. Machine learning models deployed in cybersecurity environments are highly susceptible to concept drift—a phenomenon where the statistical properties of the target variable change over time as adversaries evolve their tactics, techniques, and procedures (TTPs).¹ To combat this degradation in predictive accuracy, the ARIES architecture mandates an explicit, continuous feedback mechanism.²

The platform incorporates a Human-in-the-Loop (HITL) governance model.² When a Level-2 SOC analyst interacts with the React-based Operations UI to explicitly reject an automated containment action (e.g., preventing the isolation of a critical production server) or manually alters an alert's AI-generated triage classification from "False Positive" to "True Positive," the UI backend captures this interaction.² It immediately publishes a highly structured JSON event detailing the override and the analyst's mandatory contextual justification to a dedicated `ml.feedback` Kafka topic.²

An independent, scheduled monitoring script within the MLOps container stack continuously consumes this feedback topic. When the aggregated volume of feedback indicating false negatives or false positives crosses a predefined statistical threshold, the system automatically triggers the MLflow API.² This API call launches a localized retraining pipeline, incorporating the newly acquired, human-verified labels into the historical training corpus.² The updated model is subjected to canary validation tests against historical baselines; upon successful validation, the

new artifact is seamlessly promoted to the FastAPI inference service. This closed-loop MLOps architecture ensures that the AI models remain dynamically aligned with the highly fluid nature of contemporary cyber threats, transforming the SOC from a static defense perimeter into a continuously learning system.¹

9. Strategic Conclusion

The construction and deployment of the AI pipeline for the ARIES SOAR platform demand rigorous adherence to advanced software engineering paradigms, distributed systems architecture, and MLOps best practices. By structuring the implementation into clearly defined, heavily decoupled stages, the inherent operational risks associated with integrating artificial intelligence into mission-critical security environments are systematically mitigated.

The strategic acquisition of the Microsoft GUIDE and CTISum datasets ensures that the foundational models are trained on representative, high-fidelity security telemetry rather than highly biased synthetic approximations.⁶ Utilizing the VUB Hydra HPC cluster—specifically leveraging the immense memory bandwidth of the `ampere_gpu` partitions—provides the necessary computational velocity to train these complex architectures, while the use of localized Conda environments guarantees absolute dependency isolation and reproducibility.²³

Transitioning to the production phase, the zero-cost, localized Docker Compose stack comprising MLflow, PostgreSQL, and MinIO provides an enterprise-grade tracking and versioning system without incurring prohibitive cloud hosting fees or risking catastrophic data exfiltration.³⁵ The crucial conversion of PyTorch training artifacts to optimized ONNX formats successfully circumvents the limitations of the Python GIL, enabling the FastAPI serving layer to execute predictions with the minimal latency demanded by real-time security operations.⁵

Ultimately, wrapping this high-speed inference engine within an asynchronous aiokafka consumer loop transforms the machine learning model from a brittle, standalone Python script into a highly resilient, horizontally scalable microservice. This continuous pipeline systematically consumes, scores, and enriches telemetry, passing high-fidelity intelligence to the core Orchestration Engine and fundamentally shifting the operational capabilities of the modern Security Operations Center.²

Works cited

1. Automated Alert Classification and Triage (AACT): An Intelligent System for the Prioritisation of Cybersecurity Alerts - arXiv, accessed on February 26, 2026, <https://arxiv.org/html/2505.09843v1>
2. SRS - SOAR.pdf
3. How to Reduce Alert Fatigue in Cybersecurity: Best Practices - Prophet Security, accessed on February 26, 2026, <https://www.prophetsecurity.ai/blog/how-to-reduce-alert-fatigue-in-cybersecurity-best-practices>

4. Cybersecurity: Advanced AI-Powered - Kaggle, accessed on February 26, 2026, <https://www.kaggle.com/code/jurijsruko/cybersecurity-advanced-ai-powered>
5. FastAPI Docker Deployment: Preparing ONNX AI Models for AWS Lambda - PyImageSearch, accessed on February 26, 2026, <https://pyimagesearch.com/2025/11/17/fastapi-docker-deployment-preparing-on-nx-ai-models-for-aws-lambda/>
6. Microsoft Security Incident Prediction - Kaggle, accessed on February 26, 2026, <https://www.kaggle.com/datasets/Microsoft/microsoft-security-incident-predicti-on>
7. Large Language Models for Synthetic Dataset Generation of Cybersecurity Indicators of Compromise - PMC, accessed on February 26, 2026, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12074446/>
8. Async Pipelines for AI: How I Combine Kafka, FastAPI, and GPUs for Streaming Predictions, accessed on February 26, 2026, <https://ai.plainenglish.io/async-pipelines-for-ai-how-i-combine-kafka-fastapi-and-gpus-for-streaming-predictions-02956c5699eb>
9. Microsoft: Classifying Cybersecurity Incidents with Machine Learning - GitHub, accessed on February 26, 2026, https://github.com/udhaya2823/Microsoft---Classifying-Cybersecurity-Incidents-with-Machine_Learning
10. AI-Driven Guided Response for Security Operation Centers with Microsoft Copilot for Security - arXiv.org, accessed on February 26, 2026, <https://arxiv.org/html/2407.09017v4>
11. Microsoft_SOC_Dataset - Kaggle, accessed on February 26, 2026, <https://www.kaggle.com/datasets/avijitjana101/microsoft-soc-dataset>
12. Microsoft Guide Paper 1 | PDF | Principal Component Analysis - Scribd, accessed on February 26, 2026, <https://www.scribd.com/document/824114108/MICROSFT-GUIDE-PAPER-1>
13. Sirpi-57/Microsoft-Cyber-Security-Incident-Grade-Classification - GitHub, accessed on February 26, 2026, <https://github.com/Sirpi-57/Microsoft-Cyber-Security-Incident-Grade-Classificati-on>
14. Natural Language Processing for Threat Intelligence | Exclusive Lesson - YouTube, accessed on February 26, 2026, <https://www.youtube.com/watch?v=oPVgFYFREU4>
15. CTISum: A New Benchmark Dataset For Cyber Threat Intelligence Summarization - arXiv, accessed on February 26, 2026, <https://www.arxiv.org/pdf/2408.06576>
16. Ctisum: A New Benchmark Dataset for Cyber Threat Intelligence Summarization - SSRN, accessed on February 26, 2026, https://papers.ssrn.com/sol3/papers.cfm?abstract_id=5340605
17. CTISum: A New Benchmark Dataset For Cyber Threat Intelligence Summarization - arXiv, accessed on February 26, 2026, <https://arxiv.org/html/2408.06576v1>
18. The new dataset and baseline models for the paper CTISum: A New Benchmark Dataset For Cyber Threat Intelligence Summarization - GitHub, accessed on February 26, 2026, <https://github.com/pengwei-ii/CTISum>

19. NLP Based Cyber Security Dataset - Kaggle, accessed on February 26, 2026, <https://www.kaggle.com/datasets/hussainsheikh03/nlp-based-cyber-security-dataset/versions/1>
20. Threat Intelligence Extraction Framework (TIEF) for TTP Extraction - MDPI, accessed on February 26, 2026, <https://www.mdpi.com/2624-800X/5/3/63>
21. CTISum: A New Benchmark Dataset For Cyber Threat Intelligence Summarization - arXiv, accessed on February 26, 2026, <https://arxiv.org/html/2408.06576v2>
22. 5. Main Job Types — VUB-HPC, accessed on February 26, 2026, <https://hpc.vub.be/docs/job-submission/main-job-types/>
23. 6. GPU Job Types — VUB-HPC, accessed on February 26, 2026, <https://hpc.vub.be/docs/job-submission/gpu-job-types/>
24. Slurm Job Manager - Research Computing - The University of Virginia, accessed on February 26, 2026, <https://www.rc.virginia.edu/userinfo/hpc/slurm/>
25. 3. Slurm partitions - VUB-HPC - Vrije Universiteit Brussel, accessed on February 26, 2026, <https://hpc.vub.be/docs/job-submission/slurm-partitions/>
26. 1. Module System - VUB-HPC - Vrije Universiteit Brussel, accessed on February 26, 2026, <https://hpc.vub.be/docs/software/modules/>
27. Modules - HPC Wiki, accessed on February 26, 2026, <https://hpc-wiki.info/hpc/Modules>
28. Hugging Face Transformers (GPU) — Discoverer HPC Docs documentation, accessed on February 26, 2026, https://docs.discoverer.bg/huggingface_setup_guide.html
29. Installing specific versions of pytorch and transformers via pip - HPC Community, accessed on February 26, 2026, <http://hpc-community.unige.ch/t/installing-specific-versions-of-pytorch-and-transformers-via-pip/4065>
30. 4. Installing additional software - VUB-HPC - Vrije Universiteit Brussel, accessed on February 26, 2026, https://hpc.vub.be/docs/software/additional_software/
31. HPC training, accessed on February 26, 2026, <https://hpc.vub.be/hpctraining/>
32. Distributing Training - Hugging Face, accessed on February 26, 2026, https://huggingface.co/docs/trl/en/distributing_training
33. Using Accelerate on an HPC (Slurm) - Hugging Face Forums, accessed on February 26, 2026, <https://discuss.huggingface.co/t/using-accelerate-on-an-hpc-slurm/6286>
34. Multi-GPU Training with Hugging Face Transformers: A Complete Guide - Medium, accessed on February 26, 2026, <https://medium.com/@staytechrich/multi-gpu-training-with-hugging-face-transformers-a-complete-guide-ab2cf241df94>
35. Setting up a Development Machine with MLflow and MinIO, accessed on February 26, 2026, <https://blog.min.io/setting-up-a-development-machine-with-mlflow-and-minio/>
36. Self Hosting Overview - MLflow, accessed on February 26, 2026, <https://mlflow.org/docs/latest/self-hosting/>
37. Self Hosting Overview | MLflow, accessed on February 26, 2026, <https://mlflow.org/docs/latest/self-hosting/index.html>

38. Remote Experiment Tracking with MLflow Tracking Server, accessed on February 26, 2026, <https://mlflow.org/docs/latest/ml/tracking/tutorials/remote-server/>
39. Production-ready MLFlow setup in your local machine - Erik Dao, accessed on February 26, 2026, <https://erikdao.com/machine-learning/production-ready-mlflow-setup-in-your-local-machine/>
40. Setup collaborative MLflow with PostgreSQL as Tracking Server and MinIO as Artifact Store using docker containers | by Amir Masoud Sefidian | Medium, accessed on February 26, 2026, https://medium.com/@amir_masoud/setup-collaborative-mlflow-with-postgresql-as-tracking-server-and-minio-as-artifact-store-using-45c76a9d9814
41. Setting Up an MLflow Tracking Server for Machine Learning Experiments (Part I) - Medium, accessed on February 26, 2026, <https://medium.com/@claudia.yao2012/setting-up-an-mlflow-tracking-server-for-machine-learning-experiments-part-i-49d06262e67e>
42. Managing machine learning workflows on CSC's supercomputers - Docs CSC, accessed on February 26, 2026, <https://docs.csc.fi/support/tutorials/ml-workflows/>
43. Scaling-up PyTorch inference: Serving billions of daily NLP inferences with ONNX Runtime, accessed on February 26, 2026, <https://opensource.microsoft.com/blog/2022/04/19/scaling-up-pytorch-inference-serving-billions-of-daily-nlp-inferences-with-onnx-runtime>
44. ONNX: Train in Any Framework, Deploy on Any Hardware - DataCamp, accessed on February 26, 2026, <https://www.datacamp.com/tutorial/onnx>
45. Export to ONNX - Hugging Face, accessed on February 26, 2026, <https://huggingface.co/docs/transformers/v4.44.0/en/serialization>
46. FastAPI + GPU Magic! Real Time Anomaly Detection with ONNX CPU & CUDA Ready!, accessed on February 26, 2026, <https://www.youtube.com/watch?v=ko1LsZsyZlk>
47. Export a model to ONNX with optimum.exporters.onnx - Hugging Face, accessed on February 26, 2026, https://huggingface.co/docs/optimum-onnx/onnx/usage_guides/export_a_model
48. Export to ONNX - Hugging Face, accessed on February 26, 2026, <https://huggingface.co/docs/transformers/v4.44.0/serialization>
49. Introducing Optimum by HuggingFace | by Vinish M - Medium, accessed on February 26, 2026, <https://vinishm.medium.com/introducing-optimum-by-huggingface-52729f0065af>
50. FastAPI and Apache Kafka - Medium, accessed on February 26, 2026, <https://medium.com/@arturocuicas/fastapi-and-apache-kafka-4c9e90aab27f>
51. Simple kafka consumer and producer using fastapi | by Lemoncode21 - Medium, accessed on February 26, 2026, <https://lemoncode21.medium.com/simple-kafka-consumer-and-producer-using-fastapi-be1b55deea2>
52. AI Deploy - Tutorial - Deploy an ONNX model using FastAPI - OVH, accessed on

February 26, 2026,

https://help.ovhcloud.com/csm/en-public-cloud-ai-deploy-densenet-onnx-fastapi?id=kb_article_view&sysparm_article=KB0059789

53. FastAPI and Kafka Integration with aiokafka [Consumer and Producer] - GitHub Gist, accessed on February 26, 2026,
<https://gist.github.com/eshafik/3adb55391af1c2d94abeb826f60875a9>
54. How to add aiokafka producer to FastAPI properly? - Stack Overflow, accessed on February 26, 2026,
<https://stackoverflow.com/questions/79786387/how-to-add-aiokafka-producer-to-fastapi-properly>
55. Kafka consumers in FastAPI -> different services, different consumer, same producers, accessed on February 26, 2026,
<https://stackoverflow.com/questions/78477604/kafka-consumers-in-fastapi-different-services-different-consumer-same-producer>